CS 4150 Algorithms

Spring 2026

Assignment 6: Graph Algorithms I

Due Date: 11:59 p.m., Thursday, April 9, 2026

Note: In this assignment, we assume that all input graphs are represented by **adjacency lists**.

1. **(20 points)** Let G be a **directed** graph with n vertices and m edges, and let s be a designated vertex of G.

   (a) **(10 points)** Design an $O(m+n)$ time algorithm to determine whether the following condition holds: For every vertex $v \in G$, there exists a path from s to v in G.

   In other words, your algorithm should determine whether s can reach every vertex $v \in G$.

   (b) **(10 points)** Design an $O(m+n)$ time algorithm to determine whether the following condition holds: For every vertex $v \in G$, there exists a path from v to s in G.

   In other words, your algorithm should determine whether every vertex $v \in G$ can reach s.

   Note: The input is the adjacency lists for G. This means that all the information needed in your algorithm must be computed from the adjacency lists of G. For example, if you want to convert G to a new graph G', then you must provide an algorithm to compute the adjacency lists of G' from those of G.

   Remark (optional). Here is an application of your algorithms for (a) and (b). A directed graph G is *strongly connected* if for every pair of vertices u and v, there exist a path from u to v and a path from v to u. An important observation is that G is strongly connected if and only if, for a fixed vertex s, every vertex $v \in G$ is reachable from s and can also reach s. Using this observation, one can determine whether G is strongly connected in $O(m+n)$ time by applying your algorithms from parts (1) and (2).

2. **(20 points)** Given an undirected graph G with n vertices and m edges, suppose that s and t are two vertices in G. We already know that a shortest path from s to t can be found using the breadth-first search (BFS) algorithm. However, there may be multiple shortest paths from s to t (for an example, see Figure 1).

   Design an $O(m+n)$ time algorithm to compute the number of different shortest paths from s to t. Your algorithm does not need to list all shortest paths; it should only report their number. (Hint: Modify the BFS algorithm.)

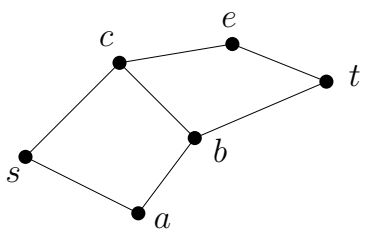

Figure 1: An example for Problem 2: There are three different shortest paths from s to t: $(s,a,b,t), (s,c,b,t), (s,c,e,t)$. So your algorithm needs to output 3 as the answer.

Remark (optional). In case you are interested, the following is an application of the above problem in social networks.

A number of art museums around the country have been featuring work by an artist named Mark Lombardi (1951–2000), consisting of a set of intricately rendered graphs. Building on a great deal of research, these graphs encode the relationships among people involved in major political scandals over the past several decades: the vertices correspond to participants, and each edge indicates some type of relationship between a pair of participants. And so, if you peer closely enough at the drawings, you can trace out ominous-looking paths from a high-ranking government official, to a former business partner, to a bank in Switzerland, to a shadowy arms dealer.

Such pictures form striking examples of *social networks*, which have vertices representing people and organizations, and edges representing relationships of various kinds. And the short paths that abound in these networks have attracted considerable attention recently, as people ponder what they mean. In the case of Mark Lombardi's graphs, they hint at the short set of steps that can carry you from the reputable to the disreputable.

Of course, a single, spurious short path between vertices s and t in such a network may be more coincidental than anything else; a large number of short paths between s and t can be much more convincing. So in addition to the problem of computing a single shortest s-t path in a graph G, social networks researchers have looked at the problem of determining the number of shortest s-t paths, which is the problem we are now considering.

3. **(20 points)** Given a **directed-acyclic-graph (DAG)** G with n vertices and m edges, let s and t be two vertices of G. There might be multiple different paths (not necessarily shortest paths) from s to t (see Figure 2 for an example).

   Design an $O(m+n)$ time algorithm to compute the number of different paths in G from s to t. Your algorithm does not need to list all paths; it should only report their number.

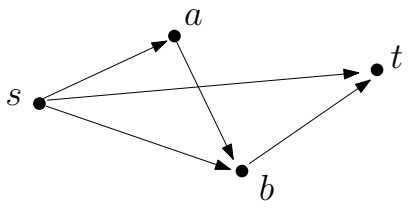

Figure 2: An example for Problem 3: There are three different paths from s to t: $s \to t$, $s \to b \to t$, and $s \to a \to b \to t$.

4. **(20 points)** In class, we studied an algorithm for computing shortest paths in directed acyclic graphs (DAGs) with edge weights. Specifically, the *length* of a path is defined as the sum of the weights of all edges on the path.

   Sometimes we are interested not only in a shortest path, but also in the number of edges in the path. For example, consider a flight graph in which each vertex is an airport, each edge represents a flight segment from one airport to another, and the weight of each edge represents the price of that flight segment. If you want to fly from airport s to airport t, you are certainly interested in a shortest path from s to t in order to minimize the total cost. However, you may also want to avoid paths with too many connections (that is, too many edges), even if they are relatively cheap. For example, suppose that you will accept only a path with at most two connections (that is, at most three edges). Then your goal is to find a shortest path from s to t subject to the constraint that the path uses at most three edges; in other words, among all paths from s to t with at most three edges, you want one with minimum total weight. This is the problem considered in this exercise.

Let G be a *directed acyclic graph* (DAG) with n vertices and m edges. Each edge (u, v) has a weight $w(u, v)$, which may be positive, zero, or negative. Let k be a positive integer given as part of the input.

A path in G is called a *k-link path* if it has at most k edges. Let s and t be two vertices of G. A *k-link shortest path* from s to t is a k-link path from s to t whose total edge weight is minimum among all k-link paths from s to t. See Figure 3 for an example.

Design an $O(k(m + n))$ time algorithm to compute a k-link shortest path from s to t. For simplicity, it is sufficient to return the length of such a path; you do not need to report the path itself.

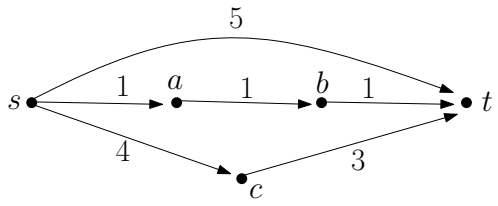

Figure 3: An example for Problem 4: Illustrating a DAG: the number besides each edge is the weight of the edge. If we are looking for a 2-link (i.e., $k = 2$) shortest path from s to t, then it is the path consisting of the only edge (s, t), whose length is 5. Although the path $s \to a \to b \to t$ is shorter, it is not a 2-link path because it has three edges. Note that the path with the only edge (s, t) is a 2-link path because it has one edge, which is indeed **no more than** two edges. There is another 2-link path from s to t: $s \to c \to t$, but it has a larger length.

Hint: Use dynamic programming and try to modify the shortest path algorithm on DAG we studied in class.

Total Points: 80